

SWIFT SHIP TRACKER USING AGENTFORCE

College Name : KSK COLLEGE OF ENGINEERING AND TECHNOLOGY

College Code:8210

Team ID : SWTID-2026-1185

S. No.	Student Name	Register No.	NM ID
1	Ashwin R	821023104006	0C25BD01A482C808C1D455D5C6CCEC33
2	Santhosh C	821023104030	D9C830EB6B69F5E9C5A4AD21637EC369
3	Sanjay D	821023104303	2B99F37A885B7A7BF3144886423071C8

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Overview

1.2 Purpose

1.3 Project Scope

1.4 Objectives

2. IDEATION PHASE

2.1 Problem Statement

2.2 User Personas

2.3 Brainstorming and Idea Prioritization

2.4 Expected User Benefits

3. REQUIREMENT ANALYSIS PHASE

3.1 Functional Requirements

3.2 Non-Functional Requirements

3.3 Data Flow Diagram

3.4 Technology Stack

3.5 Customer Journey Map

4. PROJECT DESIGN PHASE

4.1 Problem-Solution Fit

4.2 Data Model

4.3 Solution Architecture

4.4 Security Model

5. PROJECT PLANNING AND SCHEDULING

5.1 Planning Approach

5.2 Current Implementation Milestones

5.3 Implementation Sequence

6. PROJECT DEVELOPMENT AND IMPLEMENTATION

6.1 Phase 1 – Requirement Analysis and Planning

6.2 Phase 2 – Salesforce Development

6.2.1 Custom Object Configuration

6.2.2 Fields and Relationships

6.2.3 Test Data

6.3 Phase 3 – AgentForce and Security Configuration

6.3.1 Prompt Builder

6.3.2 Parcel Details Flow

6.3.3 Permission Set and Agent User

6.3.4 Swift Ship Tracker Agent and Parcel Tracking Subagent

6.3.5 Get Parcel Details Action

6.4 Phase 5 – Validation, Documentation and Maintenance

6.4.1 Validation Objectives

6.4.2 Maintenance and Troubleshooting

7. TESTING AND VALIDATION

7.1 Salesforce Data Retrieval

7.2 AgentForce Final Preview

7.3 Deployment Validation

7.4 Source Management Validation

8. CONCLUSION

9. FUTURE SCOPE

10. REFERENCES

11. APPENDIX

1. INTRODUCTION

1.1 Project Overview

Swift Ship Tracker is a Salesforce CRM solution designed to simplify parcel booking, shipment tracking, and delivery management. The supplied project reference describes a centralized platform that manages Parcel, Delivery, Sender, and Receiver information using Salesforce custom objects and relationships. The project also uses Flows for automation and AgentForce with Prompt Builder for conversational parcel information retrieval.

1.2 Purpose

- Centralize parcel and delivery information in Salesforce.
- Simplify parcel tracking through structured records and an AI-assisted interface.
- Provide a consistent parcel-tracking response using Prompt Builder.
- Automate data retrieval through Salesforce Flow.
- Apply controlled access through Salesforce security configuration.

1.3 Project Scope

The current implementation focuses on the parcel-tracking path and the Salesforce components required to support it. The evidence supplied for this report covers four custom objects, fields and relationships, test records, Prompt Builder, an Auto-Launched Flow, a Permission Set, an AgentForce Service Agent, a Parcel Tracking subagent, the Get Parcel Details action, and the final AgentForce preview. The reference material also discusses broader features such as Experience Cloud, Apex, Email Alerts, reports, and dashboards; these are retained as broader project scope or future extension unless directly evidenced by the submitted screenshots.

1.4 Objectives

- Create a clear relational data model for parcels and related parties.
- Retrieve parcel information using a Parcel ID.
- Use Flow as a reusable automation layer for parcel-detail retrieval.
- Generate a standardized conversational response using Prompt Builder.
- Connect the tracking Flow to AgentForce through a dedicated action.
- Validate the configuration through Salesforce deployment validation and functional testing.

2. IDEATION PHASE

2.1 Problem Statement

The reference identifies a set of user problems around parcel booking, tracking, delivery-status handling, communication, and operational monitoring. Customers and senders need an easier tracking experience, delivery agents need centralized operational information, support teams need parcel data for issue resolution, and administrators need visibility over the shipment process.

Problem	Primary User	Need	Effect
Parcel operations are not centralized	Customer / Sender	Simple booking and shipment management	Repeated support interactions
Shipment information is difficult to access	Customer / Sender	Quick parcel tracking	Confusion and delay
Status updates require manual handling	Delivery Agent	Centralized operational tracking	Higher manual effort
Delivery communication can be fragmented	Customer / Support	Consistent updates	Uncertainty
Operational information is scattered	Admin	Centralized reporting and visibility	Monitoring difficulty


SwiftShip Tracker
 Book. Track. Deliver. Smarter.

Problem Statements

Understanding the challenges to build a better parcel experience.

Your Parcels Our Priority

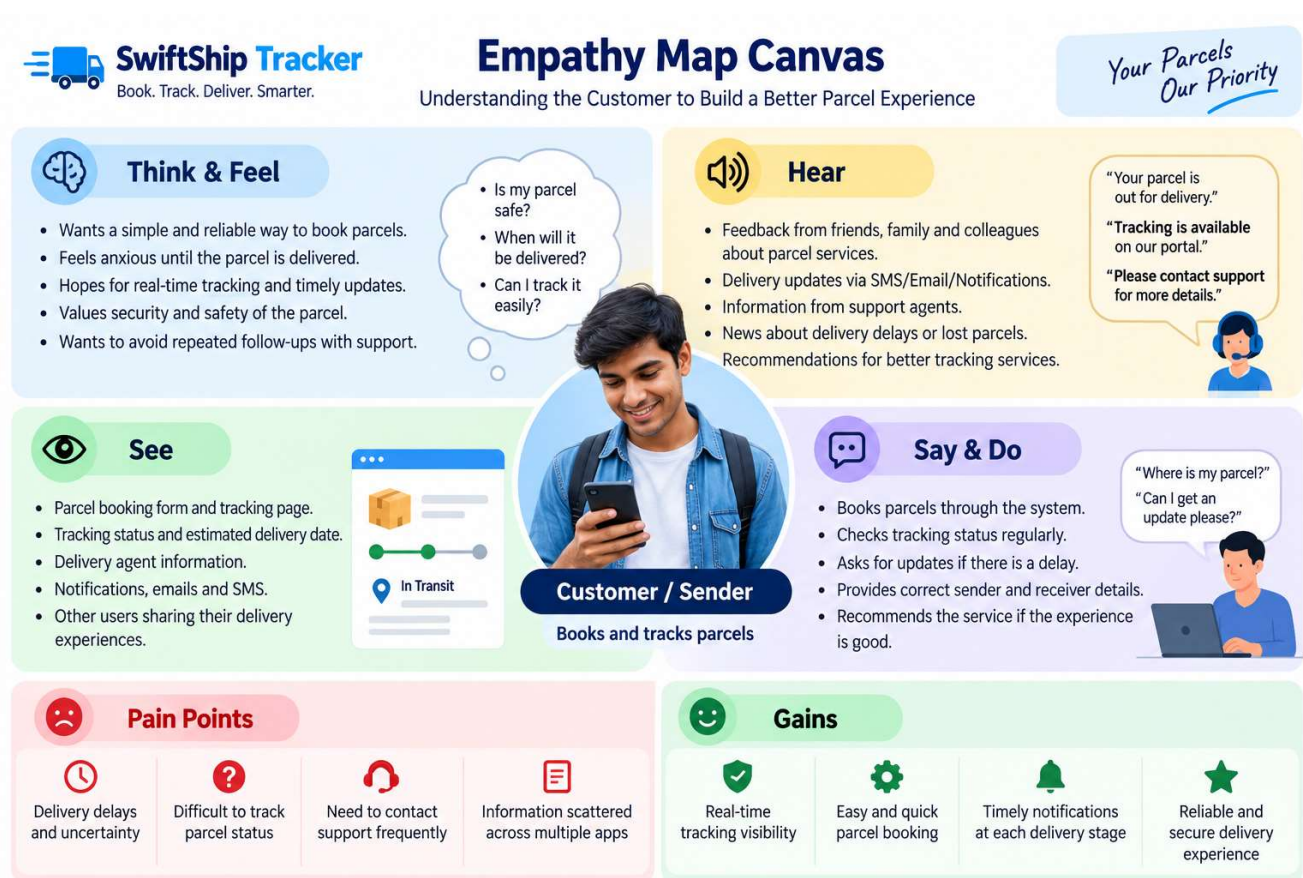
PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	 Customer / Sender	 book a parcel easily	 booking requires support assistance or multiple applications	 parcel operations are not centralized	 frustrated
PS-2	 Customer / Sender	 track my parcel	 I do not have a simple real-time tracking experience	 shipment information is difficult to access	 confused
PS-3	 Delivery Agent	 update parcel status	 status updates require manual handling	 there is a need for centralized operational tracking	 concerned
PS-4	 Customer	 receive delivery updates	 notifications may not be available at every delivery stage	 communication needs to be automated	 uncertain
PS-5	 Admin	 monitor parcel operations	 parcel flow and delivery performance are difficult to monitor	 operational data needs centralized reporting	 concerned


 Solving these challenges with Salesforce to create a simple, secure, and more connected parcel experience.


Moving Possibilities Together

2.2 User Personas

- Customer / Sender - books parcels, checks status, and expects clear shipment information.
- Delivery Agent - updates delivery status and monitors assigned parcels.
- Customer Support Team - handles delivery-related questions and exceptions.
- Admin Team - manages Salesforce configuration, security, and operational monitoring.
- System Integrator / Developer - implements automation, AI agent setup, and integrations.

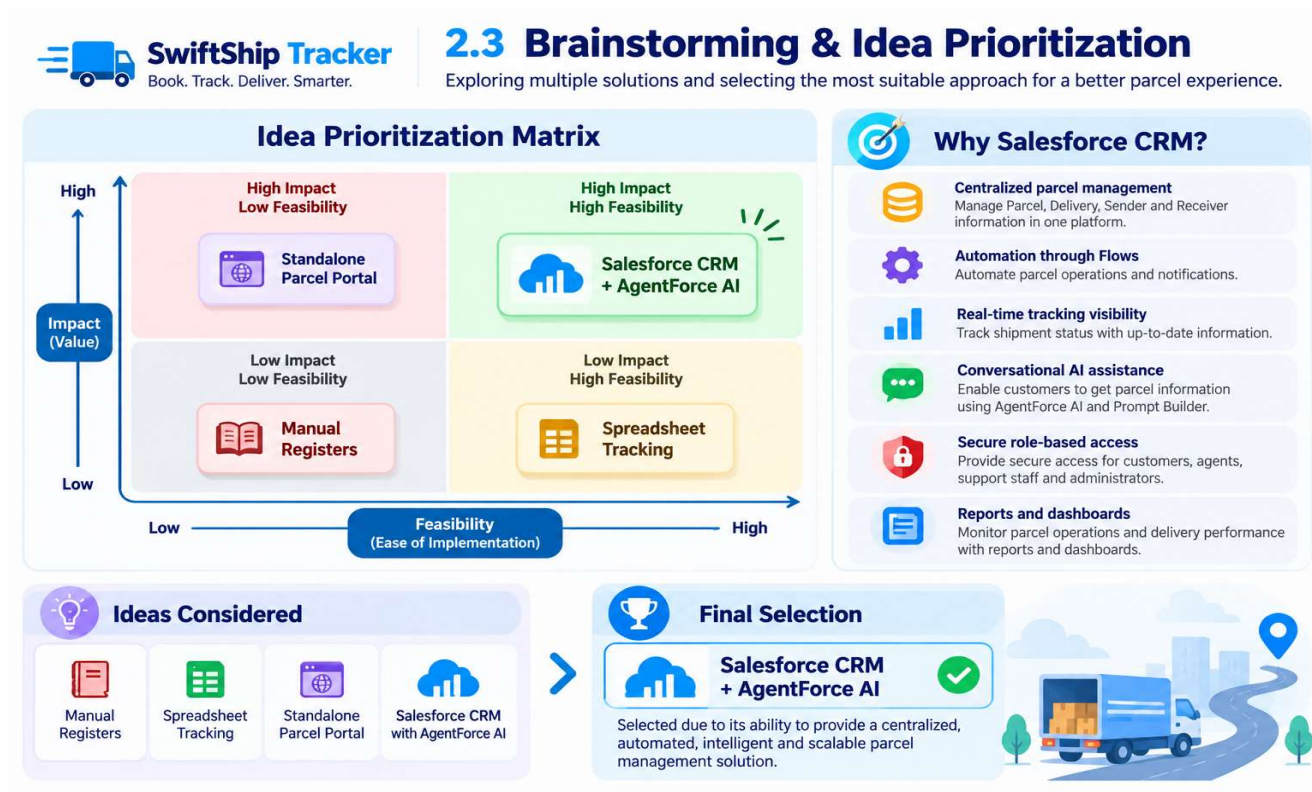


2.3 Brainstorming and Selection

The reference considers manual registers, spreadsheet tracking, a standalone parcel portal, and a Salesforce CRM approach with AgentForce AI. The selected direction uses Salesforce as the central data and automation platform and adds conversational AI for self-service tracking. This combination keeps parcel data and the tracking response within the same platform.

2.4 Expected User Benefits

- Faster access to parcel information.
- Reduced dependency on manual lookup for basic tracking questions.
- Consistent parcel status presentation.
- Centralized handling of parcel, sender, receiver, and delivery information.
- A foundation for extending the system with reporting and external logistics integrations.



3. REQUIREMENT ANALYSIS PHASE

3.1 Functional Requirements

Requirement ID	Requirement
FR-1	Parcel booking and record creation
FR-2	Parcel status tracking
FR-3	Weight management
FR-4	Estimated delivery date management
FR-5	Sender information management
FR-6	Receiver information management
FR-7	Delivery and current-location tracking
FR-8	Agent parcel status updates
FR-9	Automated customer notifications
FR-10	AI-powered parcel tracking
FR-11	Operational reports and dashboards
FR-12	Customer self-service access through Experience Cloud as a future extension

3.2 Non-Functional Requirements

NFR	Description
Usability	The user interface and tracking response should be clear and easy to understand.
Security	Access should be controlled through Salesforce permissions and field-level visibility.
Reliability	Parcel status and delivery information should be returned from Salesforce records.
Performance	Flow and AgentForce interactions should respond within acceptable time for a prototype.
Availability	The solution is hosted in Salesforce cloud infrastructure.

NFR	Description
Scalability	The object-based design should support additional parcel records and operations.
Maintainability	Flows, prompts, topics, actions, and permissions should be updateable.

3.3 Data Flow Diagram

The implemented parcel-tracking data path is represented below. A natural-language query is routed to the Parcel Tracking subagent, which invokes the Get Parcel Details action. The action starts the Parcel Details Flow, retrieves the matching Parcel record, formats the response through the configured prompt action, and returns the result to the agent.

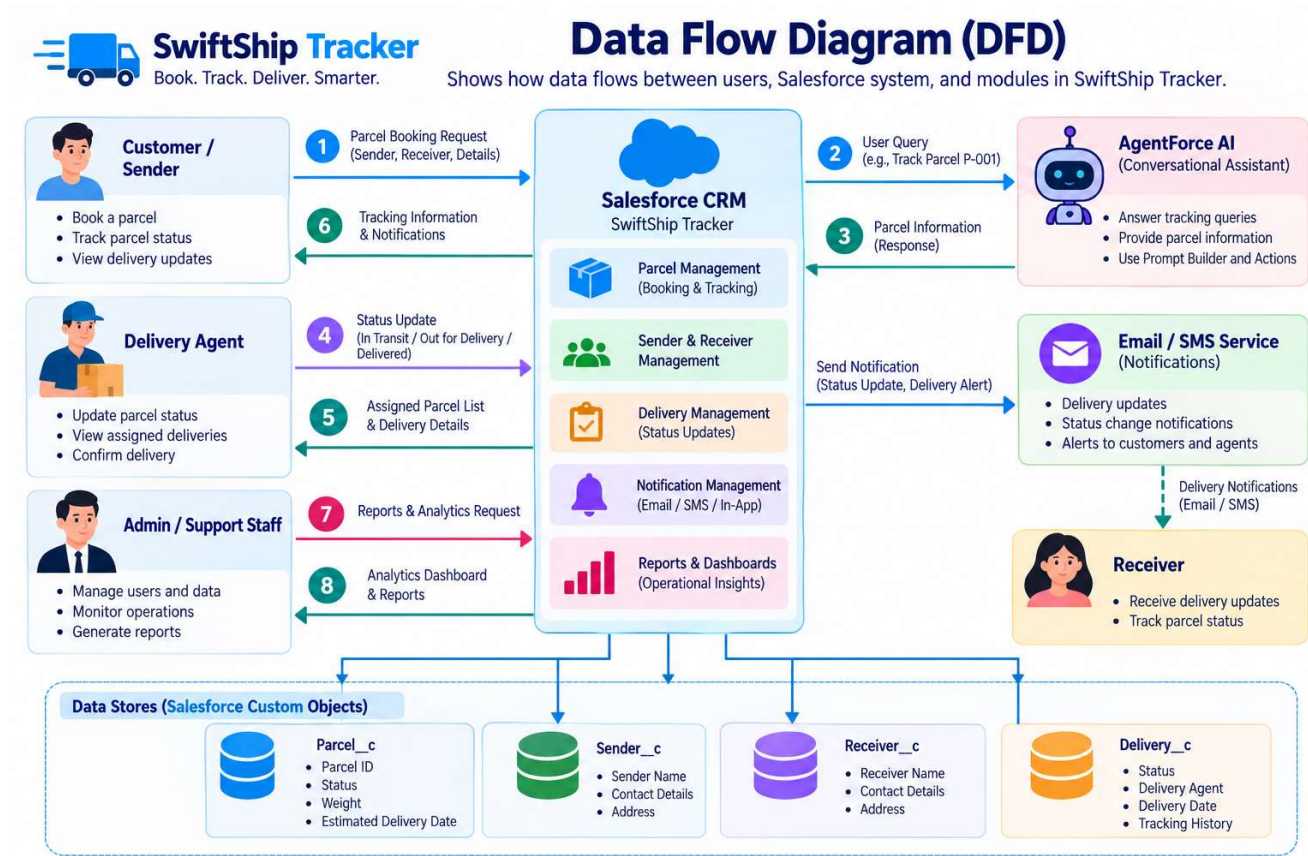


Figure 1. Implemented parcel-tracking workflow used by the current prototype.

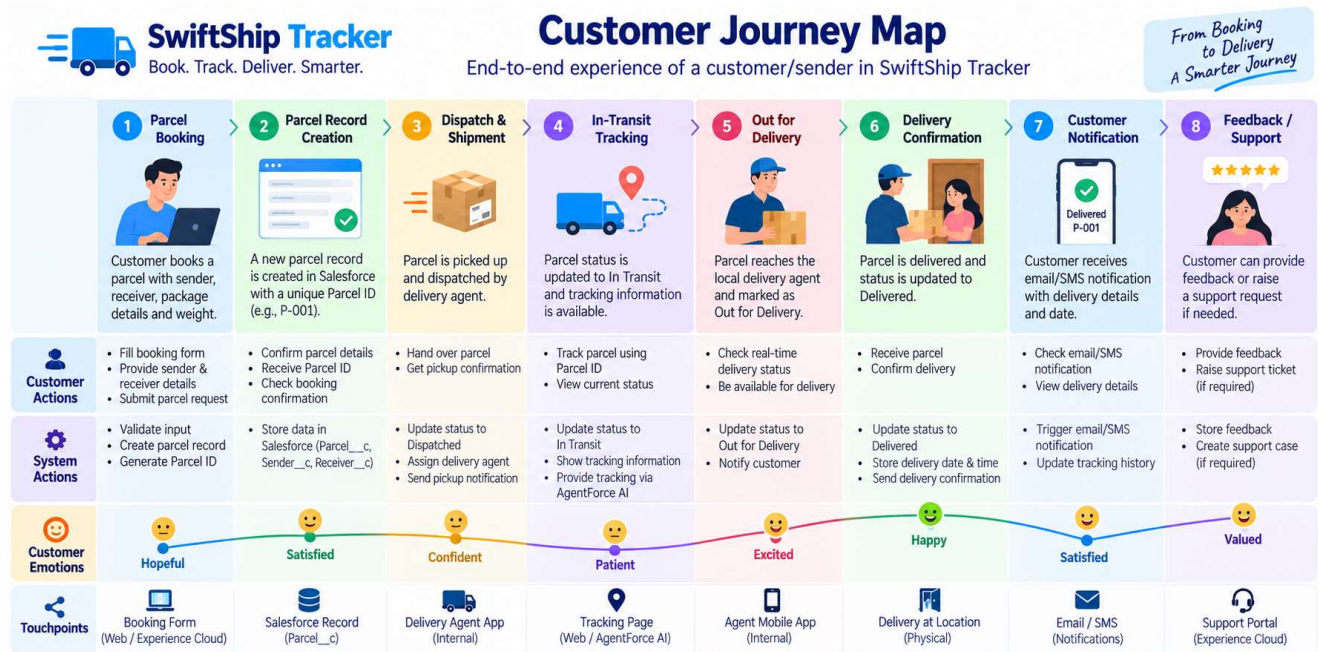
3.4 Technology Stack

Layer	Technology / Component
Platform	Salesforce Developer Edition
Data Layer	Salesforce Custom Objects: Parcel__c, Delivery__c, Sender__c, Receiver__c
Business Logic	Salesforce Flow Builder
Automation	Auto-Launched Flow and Prompt Builder action
AI	AgentForce AI and Prompt Builder
Security	Permission Set and Agent User profile/access configuration
Validation	Salesforce CLI metadata deployment validation and data query
Source Management	Git and GitHub

3.5 Customer Journey Map

Stages:

- Parcel Booking
- Parcel Record Creation
- Dispatch & Shipment
- In-Transit Tracking
- Out for Delivery
- Delivery Confirmation
- Customer Notification
- Feedback / Support



4. PROJECT DESIGN PHASE

4.1 Problem-Solution Fit

Problem	Solution in Current Prototype	Outcome
Tracking data needs structure	Parcel, Delivery, Sender and Receiver objects	Organized relational shipment data
Tracking answers should be consistent	Retrieve Parcel Details Prompt Builder template	Standardized output format
Parcel lookup should be reusable	Parcel Details Auto-Launched Flow	Reusable retrieval logic
Users need conversational tracking	Swift Ship Tracker Agent + Parcel Tracking subagent	Natural-language tracking interaction
Agent needs controlled data access	Swift Ship Permission Set + agent permissions	Configured access for the agent context



4.2 Data Model

Object	Purpose	Key Fields / Relationships
Parcel__c	Core shipment record	Parcel ID, Status, Weight, Estimated Delivery, Sender lookup
Delivery__c	Delivery-specific information	Current Location, Estimated Delivery, Sender lookup, Parcel lookup
Sender__c	Sender information	Sender Address, Contact, Email
Receiver__c	Receiver information	Receiver Address, Contact, Email, Sender lookup, Parcel lookup

The reference design places Parcel at the center of the shipment information model and connects it with Sender, Receiver, and Delivery records. The implementation screenshots show the corresponding Salesforce objects, fields, and sample records.

4.3 Solution Architecture

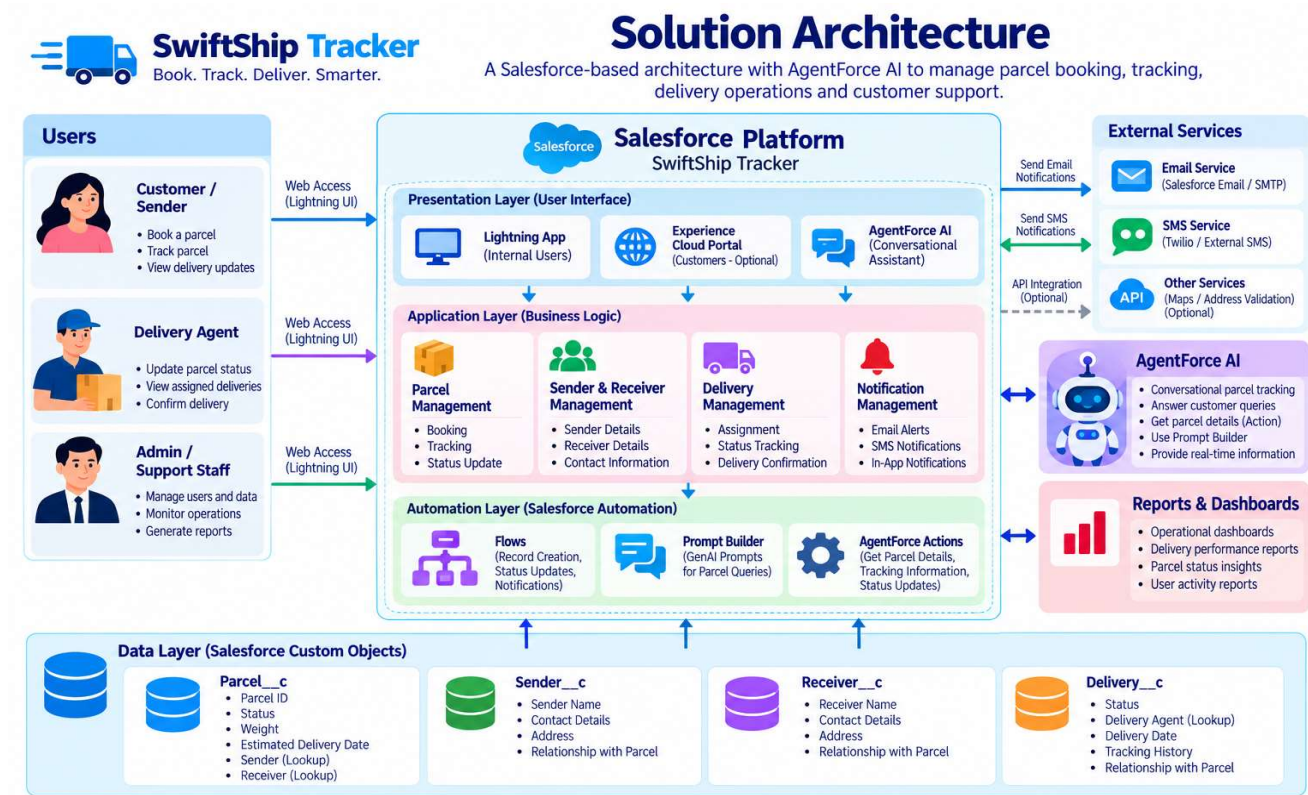


Figure 22. Implemented architecture showing users, Salesforce data, automation, AI, security, and validation layers.

4.4 Security Model

- A dedicated Swift Ship Permission Set was configured for Parcel, Delivery, Sender, and Receiver access.
- Relevant fields were made visible for the working agent configuration.
- The Einstein Agent User profile and associated AgentForce permission sets were configured for the agent user.
- The project follows the reference principle of role-based and field-level control, while the current screenshots specifically evidence the agent-user permission configuration.

5. PROJECT PLANNING AND SCHEDULING

5.1 Planning Approach

The reference project uses an Agile approach with epics, user stories, story points, sprint-based execution, and velocity estimation. For the current implementation, the practical sequence followed was: Salesforce org setup, object creation, tabs and fields, test data, Prompt Builder, Flow, permissions, AgentForce, validation, GitHub source management, and documentation.

5.2 Current Implementation Milestones

Stage	Implementation Focus	Evidence in This Report
Phase 1	Requirement analysis and scope	Chapters 1-3
Phase 2	Salesforce backend and configuration	Figures 2-17
Phase 3	Permissions and AgentForce configuration	Figures 18-22
Phase 5	Testing, deployment validation, documentation and maintenance	Chapter 7 + Appendix

5.3 Implementation Sequence

1. Create Salesforce custom objects: Parcel, Delivery, Sender, Receiver.
2. Create tabs and configure object navigation.
3. Create custom fields and relationships.
4. Create Sender, Receiver, Parcel, and Delivery test data.
5. Create and test Retrieve Parcel Details in Prompt Builder.
6. Create and activate the Parcel Details Flow.
7. Configure Permission Set and Agent User permissions.
8. Build Swift Ship Tracker Agent and Parcel Tracking subagent.
9. Create Get Parcel Details action connected to the Flow.
10. Activate and preview the agent.
11. Validate the metadata and synchronize the Salesforce source to GitHub.

6. PROJECT DEVELOPMENT AND IMPLEMENTATION

6.1 Phase 1 - Requirement Analysis and Planning

The project scope was defined around centralized parcel management and conversational tracking. The intended lifecycle described in the reference is booking -> dispatch -> tracking -> delivery confirmation. For the implemented prototype, the evidence concentrates on the tracking portion of this lifecycle and on the underlying records needed by the agent.

6.2 Phase 2 - Salesforce Development

6.2.1 Custom Object Configuration

Four custom objects were created for the project. Each object was configured in Salesforce Object Manager and deployed in the Developer Org. The screenshots below provide the project-specific evidence.

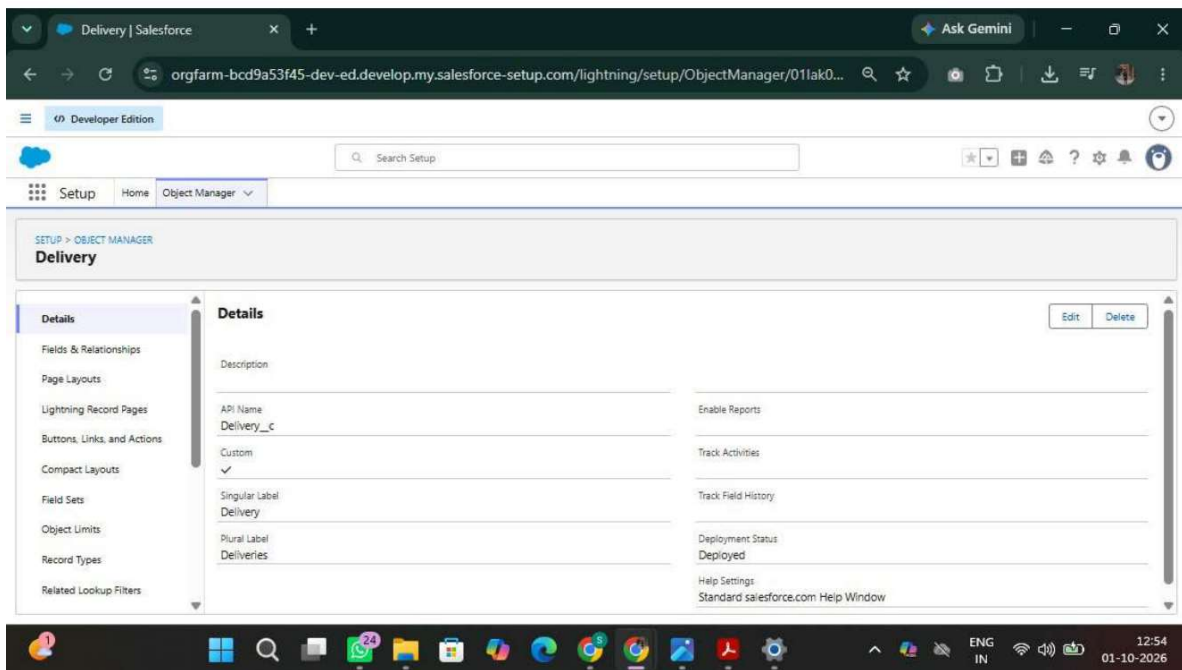


Figure 2. Delivery custom object configuration in Salesforce Object Manager.

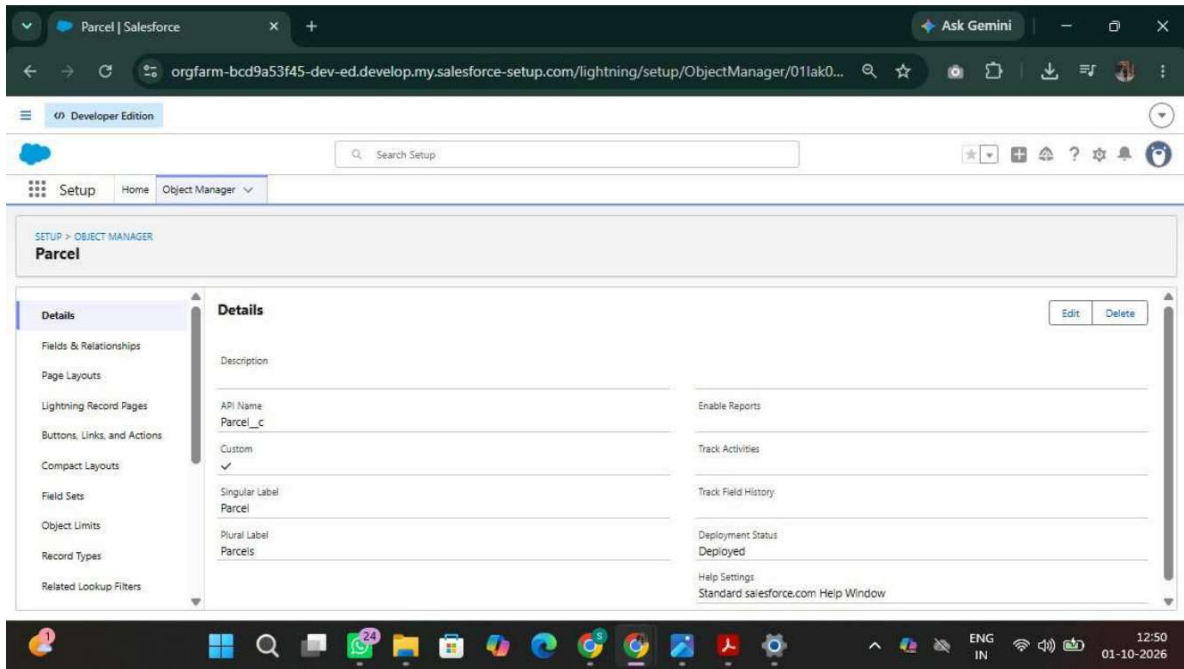


Figure 3. Parcel custom object configuration in Salesforce Object Manager.

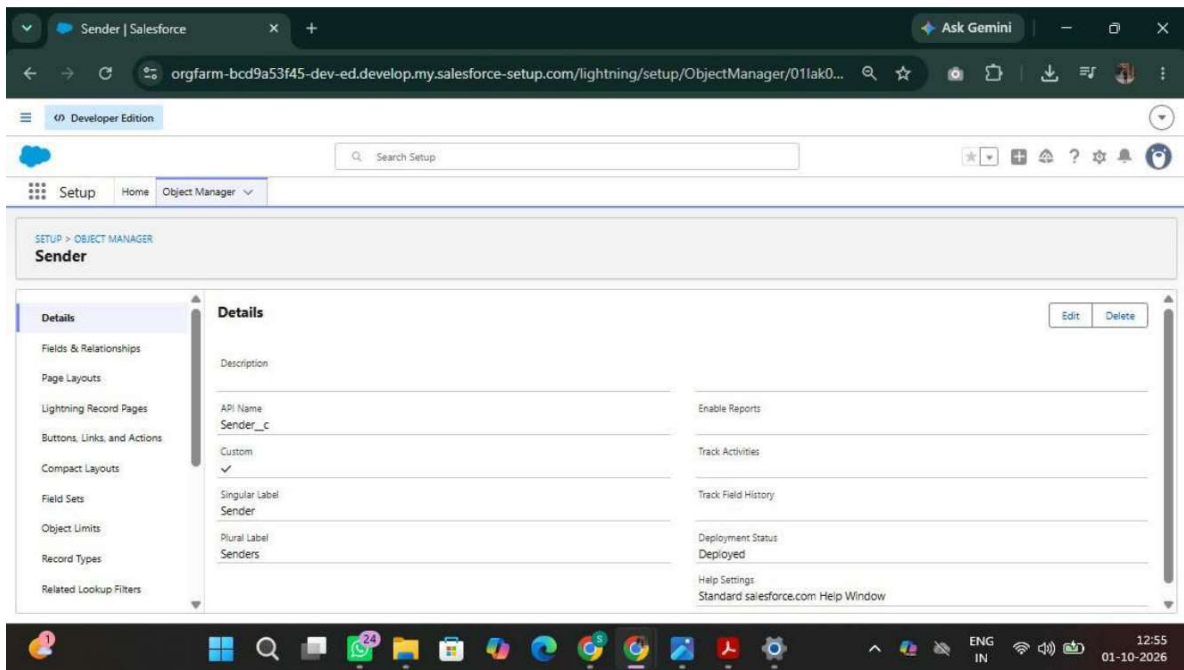


Figure 4. Sender custom object configuration in Salesforce Object Manager.

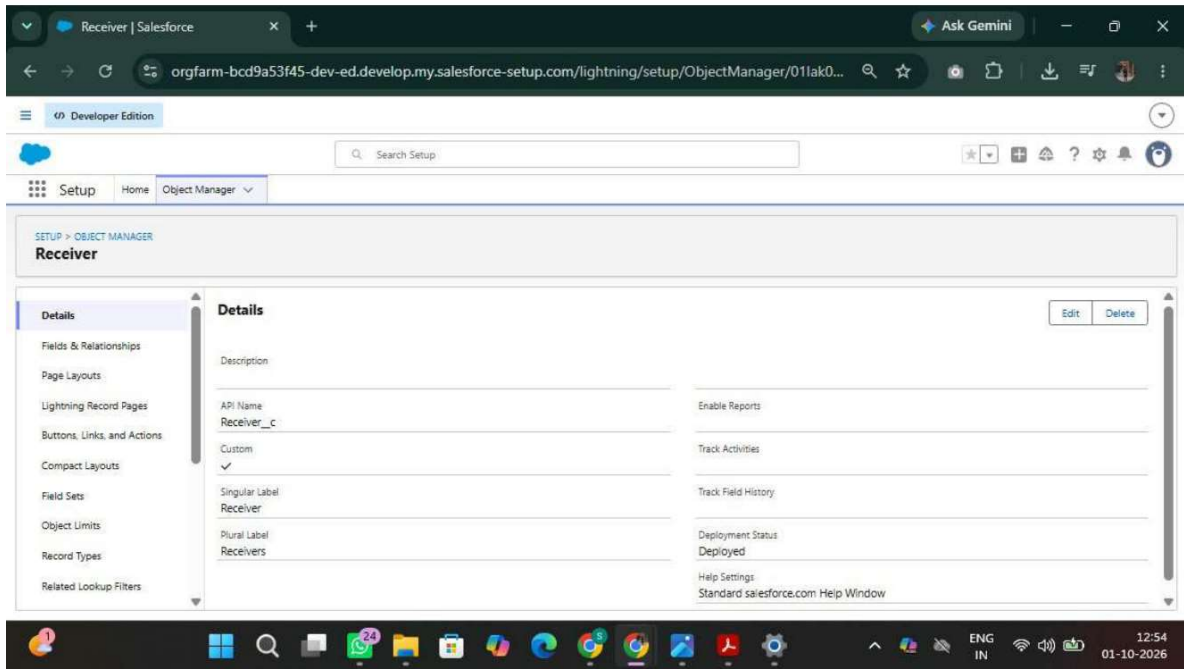


Figure 5. Receiver custom object configuration in Salesforce Object Manager.

6.2.2 Fields and Relationships

Object	Field	Type	Purpose
Parcel__c	Parcel ID	Auto Number (P-{000})	Unique parcel identifier
Parcel__c	Status	Picklist	Booked / In Transit / Out of Delivery / Delivered
Parcel__c	Weight	Number	Stores parcel weight
Parcel__c	Estimated Delivery	Date	Expected delivery date
Parcel__c	Sender	Lookup	Links the parcel to its sender
Delivery__c	Current Location	Geolocation	Stores delivery location
Delivery__c	Estimated Delivery	Date	Expected delivery date
Delivery__c	Sender	Lookup	Links delivery to sender
Delivery__c	Parcel	Lookup	Links delivery to parcel
Sender__c	Sender Address	Geolocation	Stores sender location
Sender__c	Sender Contact	Phone	Sender contact number
Sender__c	Sender Email	Email	Sender email

Receiver fields and their relationships are documented separately so the complete field table remains readable and aligned on the page.

Object	Field	Type	Purpose
Receiver__c	Receiver's Address	Geolocation	Stores receiver location
Receiver__c	Receiver Contact	Phone	Receiver contact number
Receiver__c	Receiver Email	Email	Receiver email
Receiver__c	Sender	Lookup	Associates receiver with sender
Receiver__c	Parcel	Lookup	Associates receiver with parcel

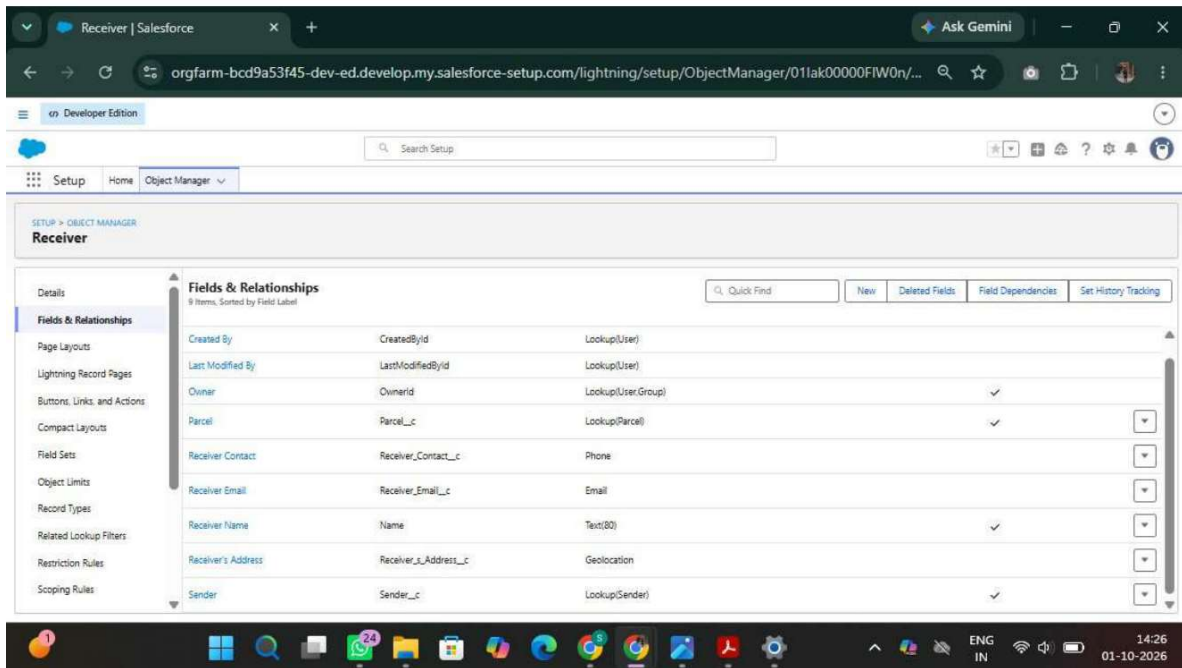


Figure 6. Receiver fields and relationships configured in Salesforce.

SWIFT SHIP TRACKER | PROJECT DOCUMENTATION

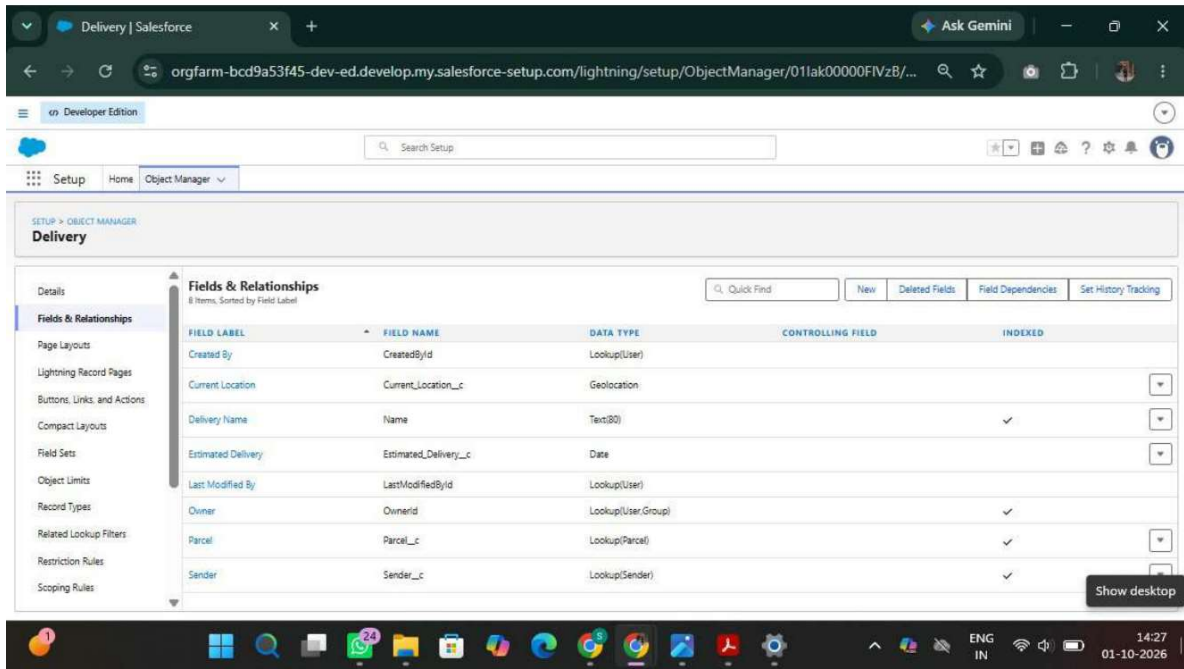


Figure 7. Delivery fields and relationships configured in Salesforce.

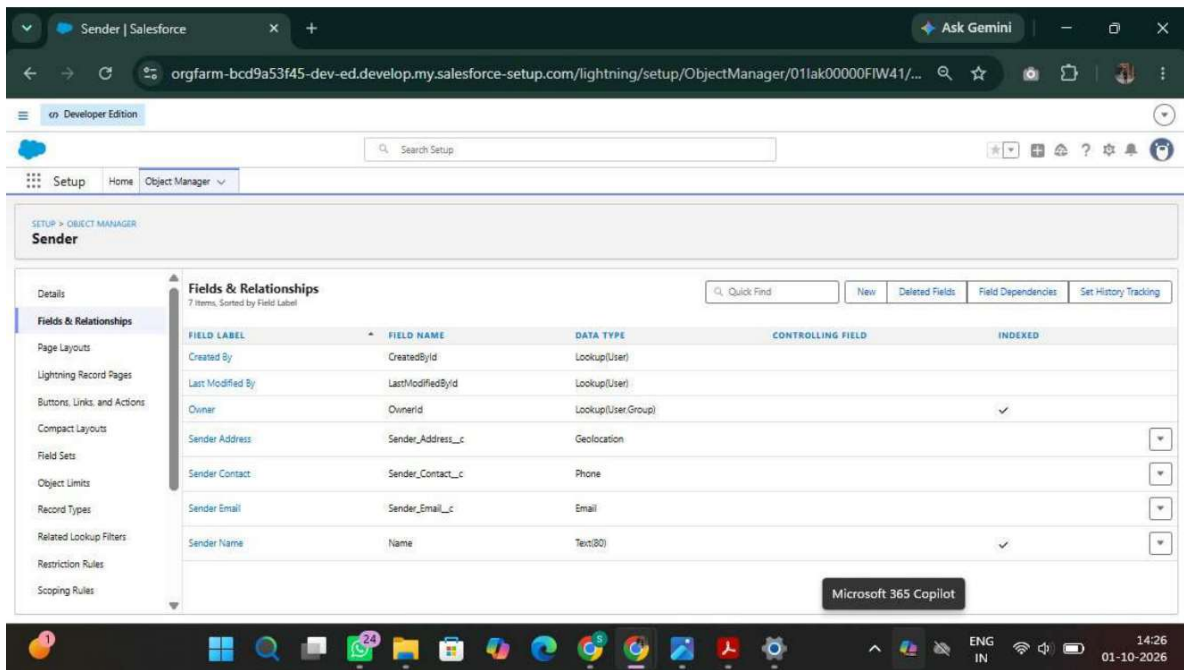


Figure 8. Sender fields and relationships configured in Salesforce.

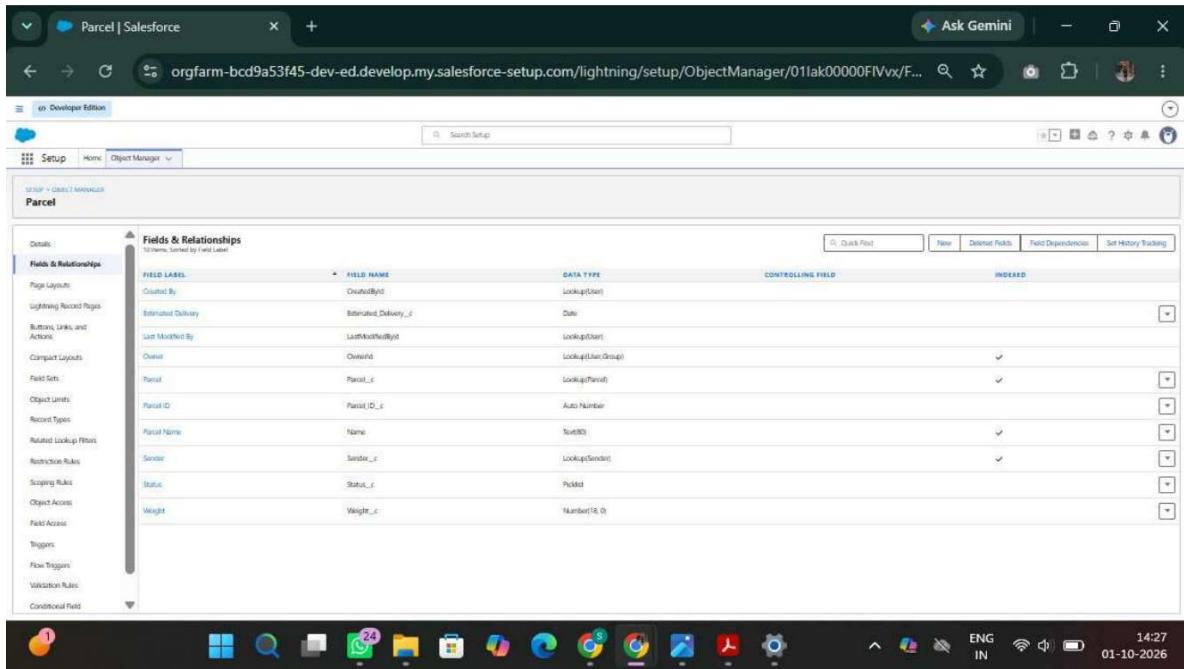


Figure 9. Parcel fields and relationships configured in Salesforce.

6.2.3 Test Data

The implementation uses project-specific test records to demonstrate the object relationships. The main records shown in the submitted evidence are FUTECH Technologies as the sender, Ashwin Customer as the receiver, FUTECH Laptop Package as the parcel, and Delivery - P-001 as the delivery record.

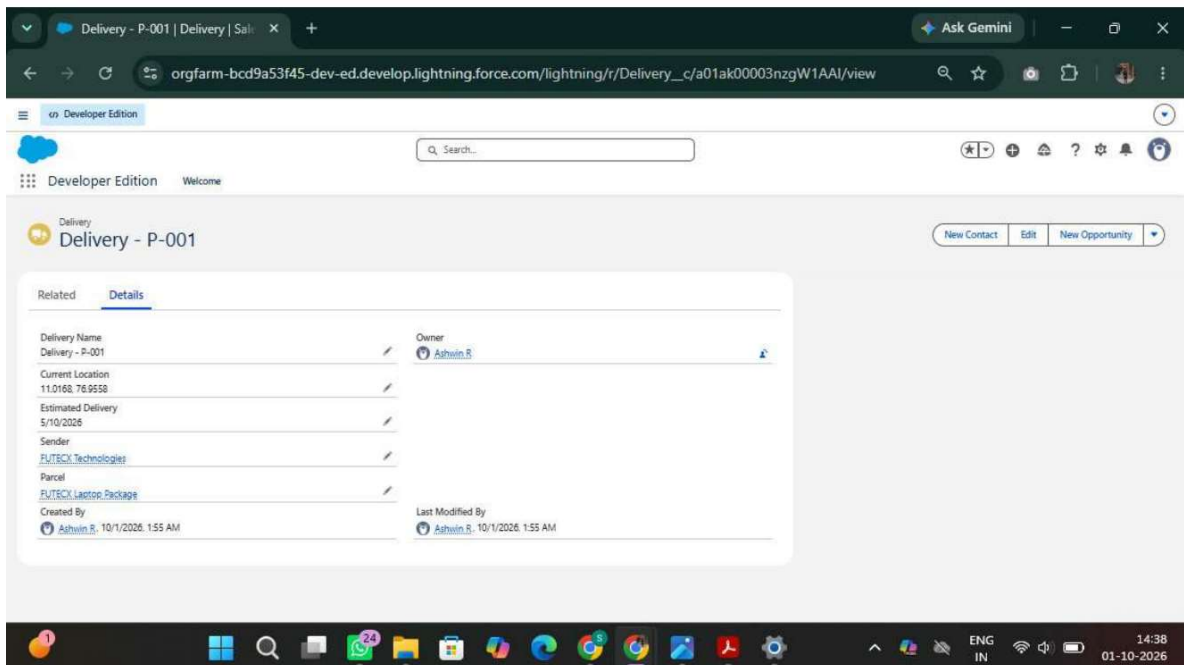


Figure 10. Delivery record linked to parcel P-001.

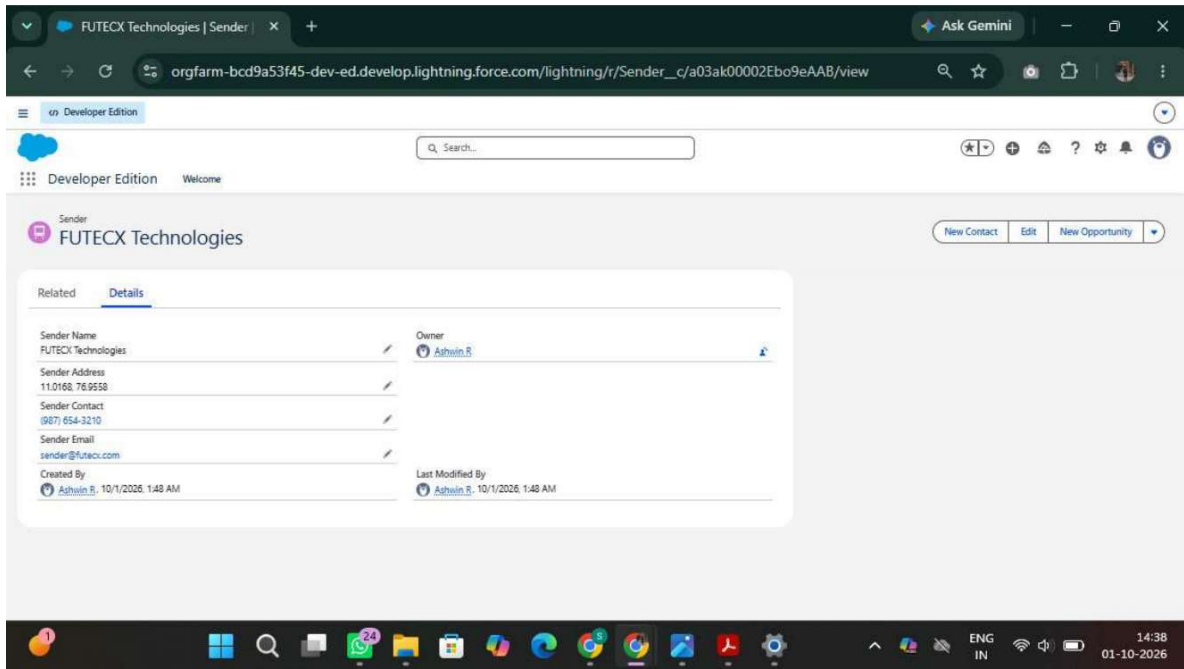


Figure 11. Sender record for FUTECX Technologies.

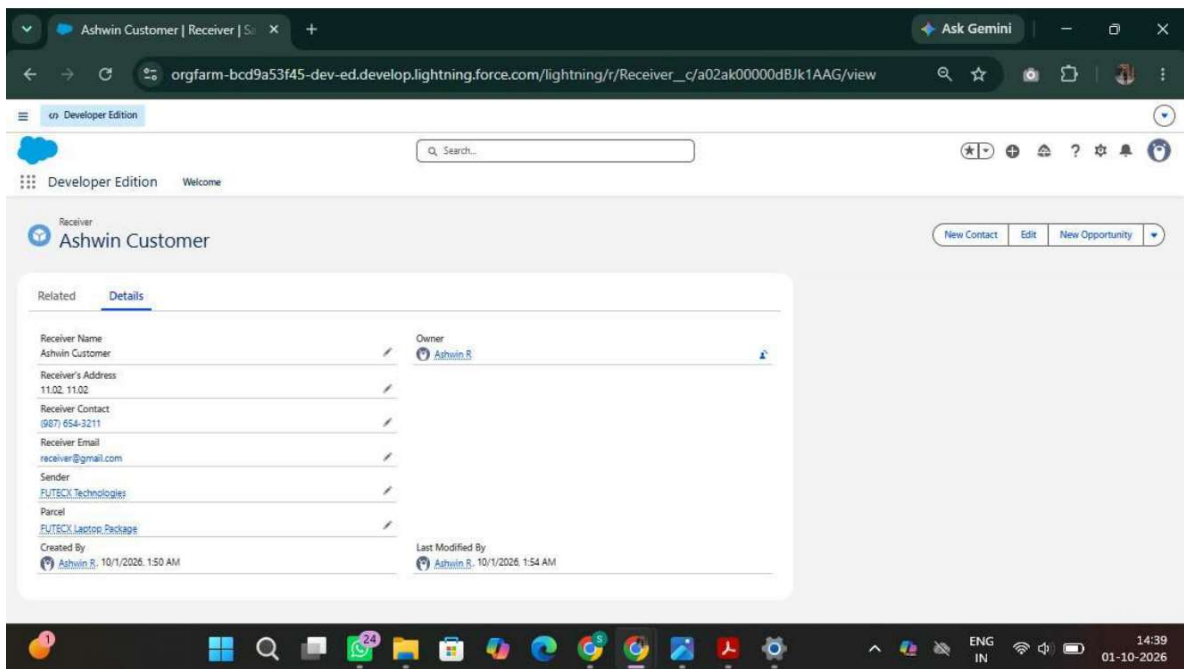


Figure 12. Receiver record for Ashwin Customer.

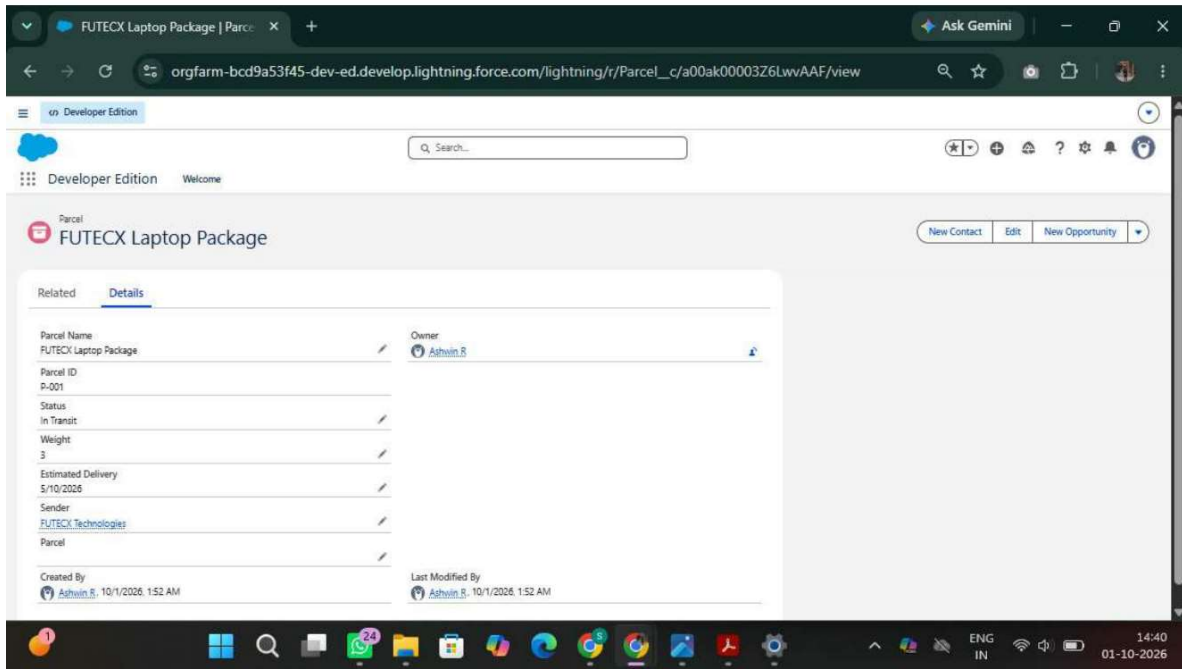


Figure 13. Parcel record for FUTECX Laptop Package, showing P-001 and tracking fields.

6.3 Phase 3 - AgentForce and Security Configuration

6.3.1 Prompt Builder

A Record Summary Prompt Template was created with the label Retrieve Parcel Details and API name Retrieve_Parcel_Details. Its purpose is to retrieve parcel details from the Salesforce Parcel record and return a consistent tracking response. The project-specific prompt references Parcel Name, Parcel ID, Status, Weight, and Estimated Delivery Date.

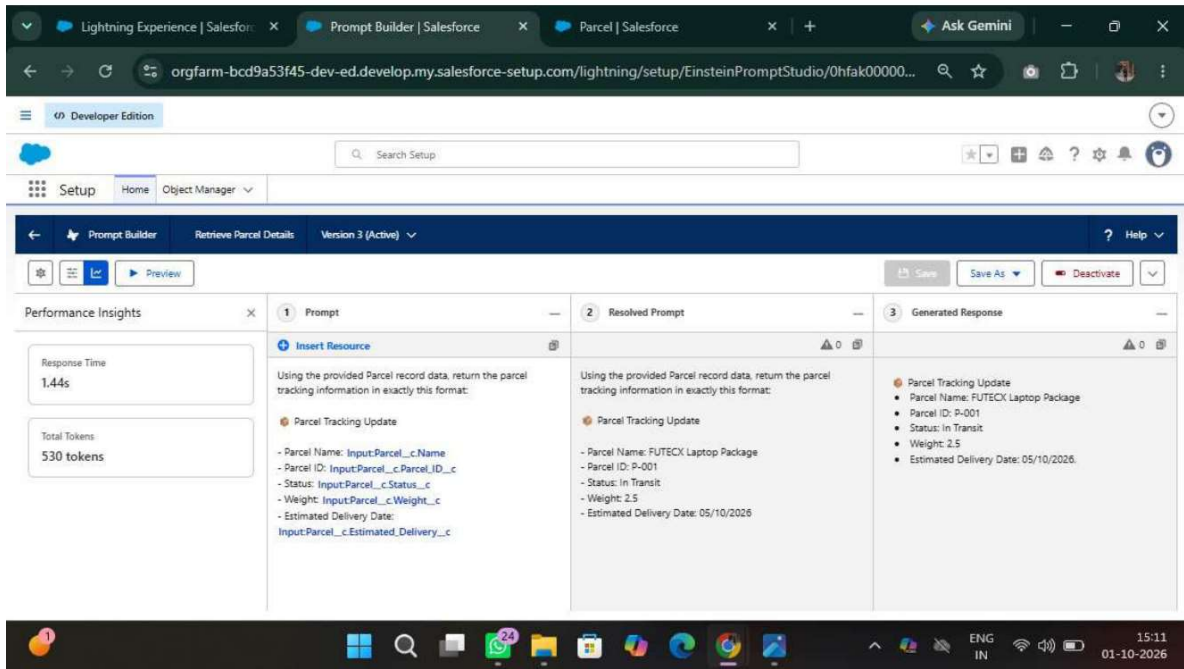


Figure 14. Prompt Builder execution for Retrieve Parcel Details, showing the generated parcel tracking response.

6.3.2 Parcel Details Flow

The implemented Flow is an Auto-Launched Flow named Parcel Details. It defines an input variable `ids` and an output variable `output`. The flow retrieves the first matching Parcel record where `Parcel_ID__c` equals the provided input, invokes the Retrieve Parcel Details prompt action, and assigns the prompt response to the output variable.

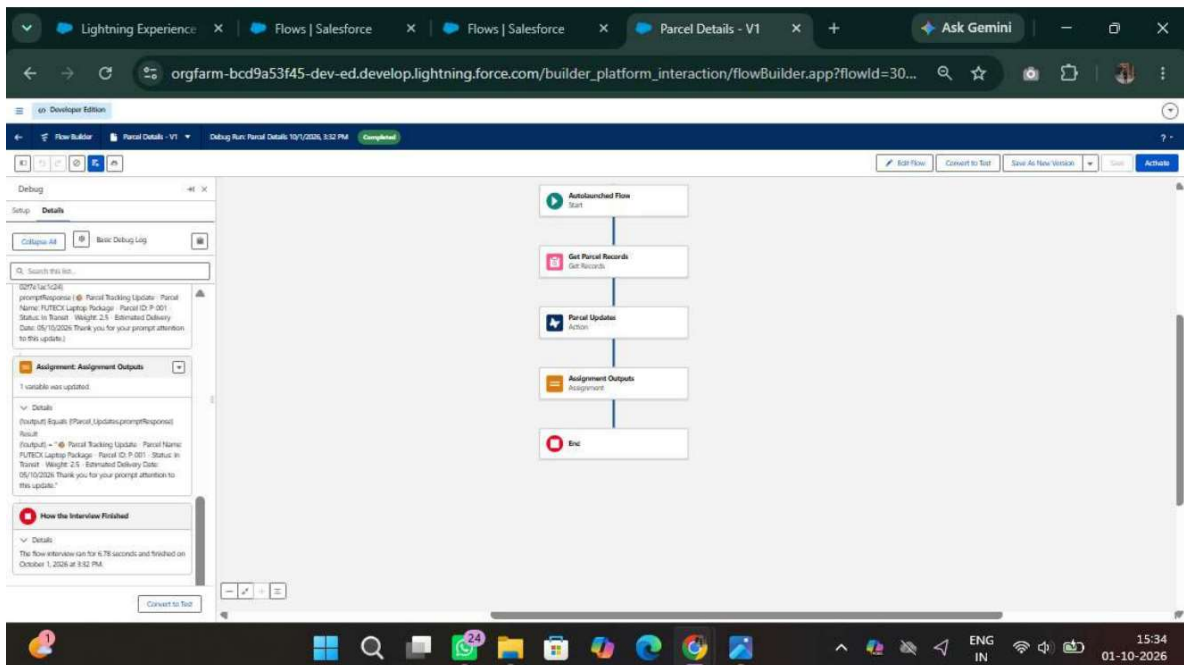


Figure 15. Parcel Details Flow showing the primary sequence from Start to Get Parcel Records.

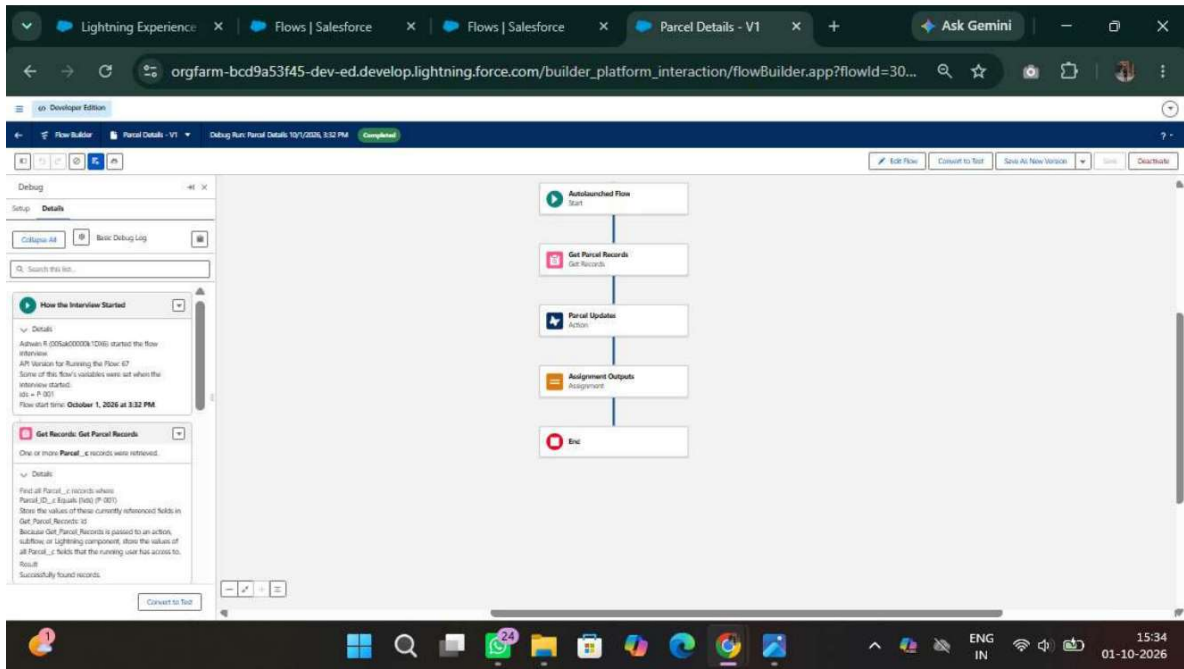


Figure 16. Parcel Details Flow with configured Get Records and prompt-response elements visible.

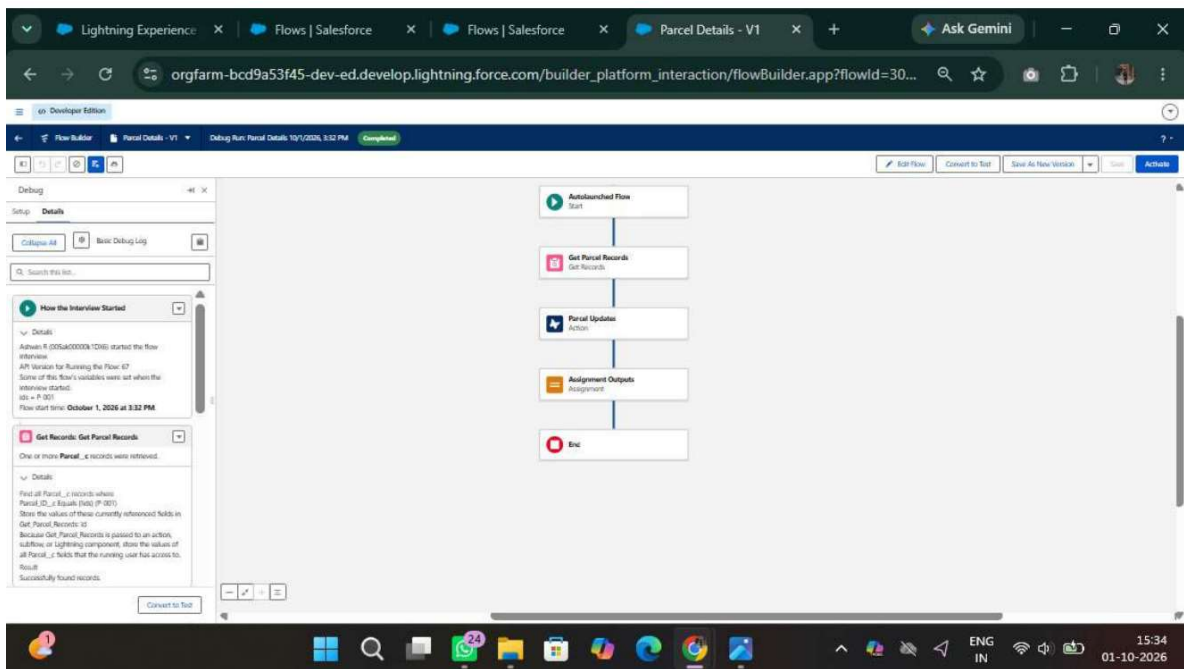


Figure 17. Parcel Details Flow saved configuration and execution structure.

6.3.3 Permission Set and Agent User

The Swift Ship Permission Set was configured to provide access to the four custom objects used by the agent workflow. The Agent User was configured with the Salesforce Einstein Agent User profile and associated AgentForce permission assignments.

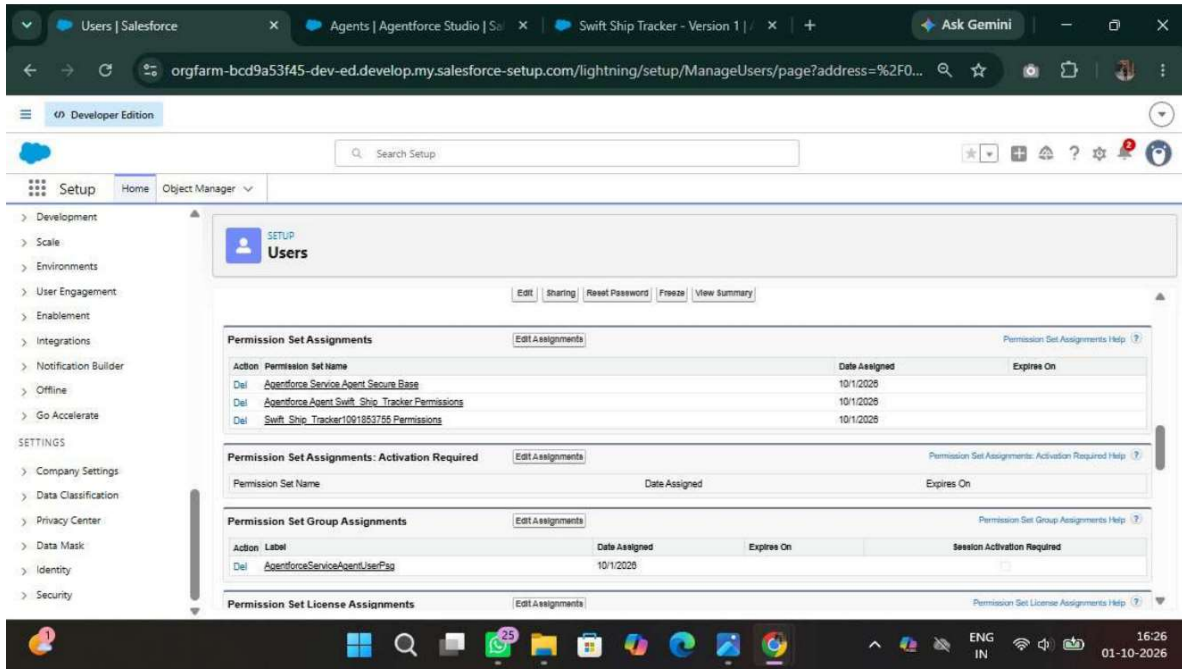


Figure 18. Agent user permission-set assignments in Salesforce Setup.

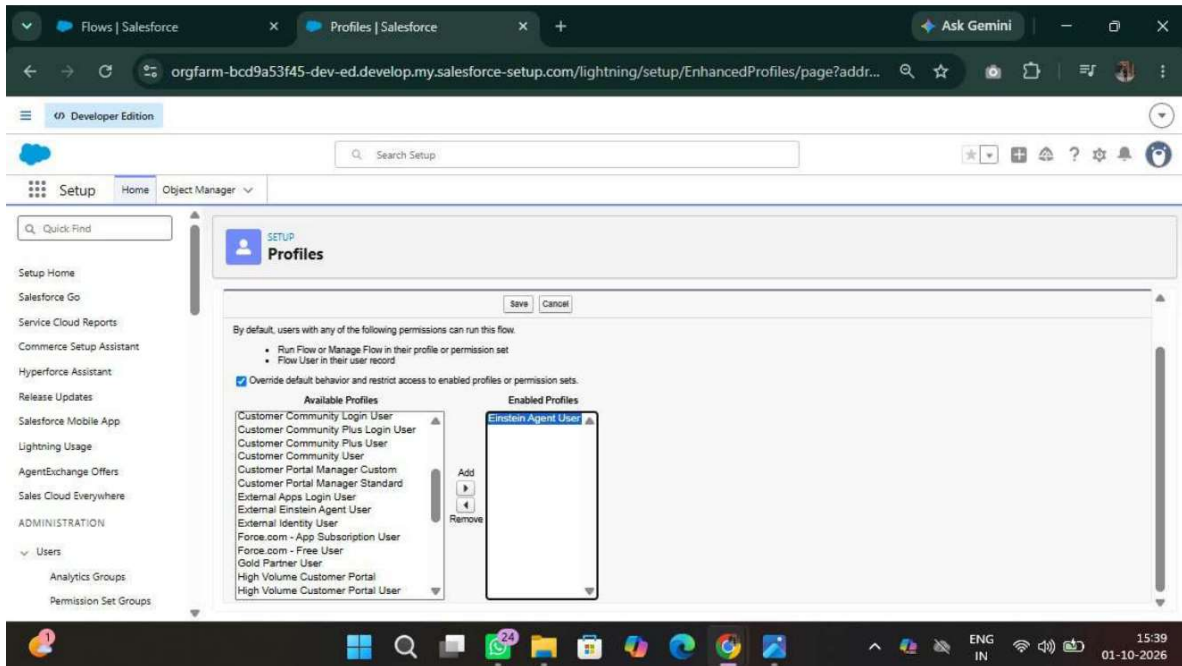


Figure 19. Einstein Agent User profile access configuration.

6.3.4 Swift Ship Tracker Agent and Parcel Tracking Subagent The AgentForce configuration contains the Swift Ship Tracker Service Agent and a Parcel Tracking subagent. The Parcel Tracking subagent is intended to handle requests for parcel tracking information. Its reasoning instructions require the user to provide a Parcel ID when one is not present, invoke the Get Parcel Details action, return the retrieved parcel information, and avoid inventing parcel data.

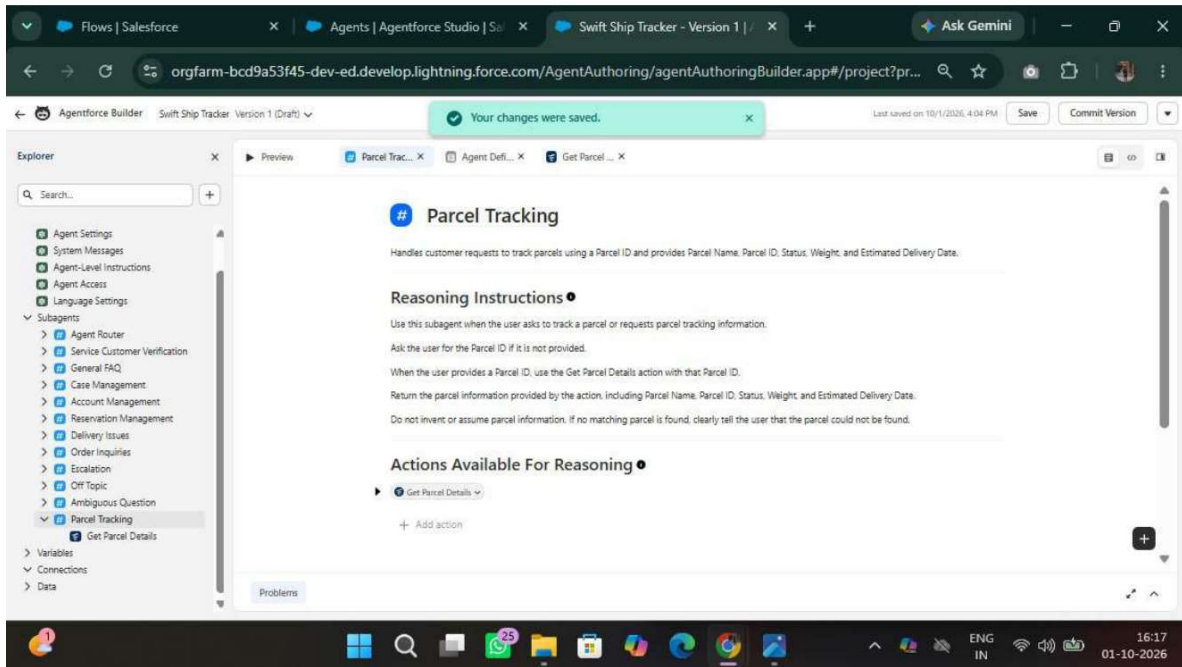


Figure 20. Parcel Tracking subagent configuration with reasoning instructions and available action.

6.3.5 Get Parcel Details Action

The custom AgentForce action is named Get Parcel Details and is backed by the Parcel Details Flow. The action accepts the ids input, passes it to the Flow, and returns the output string containing the formatted parcel tracking response. User confirmation is not required for this retrieval action, and the output is shown in the conversation.

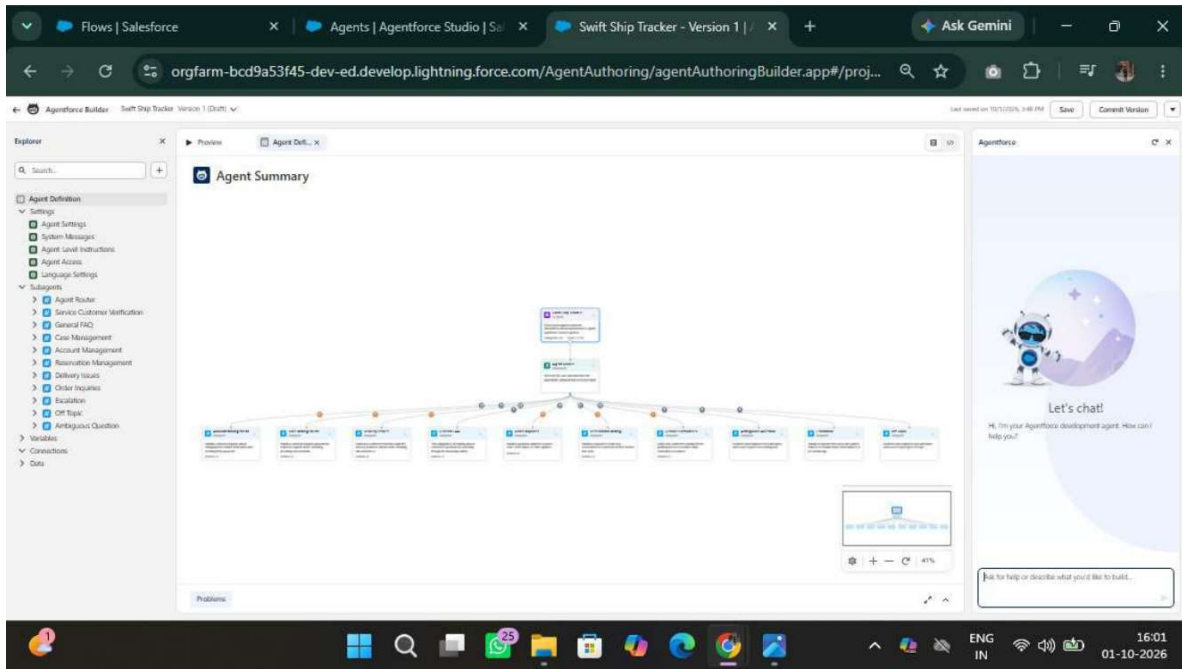


Figure 21. Swift Ship Tracker Agent Summary showing the configured agent structure and topics.

6.4 Phase 5 - Validation, Documentation and Maintenance

The reference defines Phase 5 as Deployment, Documentation and Maintenance and emphasizes validating topic recognition, Flow execution, and data retrieval inside the Salesforce Developer Org. The same reference notes that enterprise UAT, sandbox staging, and CI/CD deployment are outside the scope of this academic prototype.

6.4.1 Validation Objectives

- Verify that the Parcel Tracking subagent is recognized for a tracking query.
- Verify that the Get Parcel Details action invokes the Parcel Details Flow.
- Verify that the Flow retrieves the correct Parcel record by Parcel ID.
- Verify that the response contains the required parcel information.
- Verify that the project metadata can be validated through Salesforce CLI.

6.4.2 Maintenance and Troubleshooting

- Keep Parcel, Delivery, Sender, and Receiver data accurate and synchronized.
- Monitor for failed Flow executions or incorrect routing to the Parcel Tracking subagent.
- Update Flow logic when parcel processes change.
- Refine the Prompt Builder template when response requirements change.
- Review permission assignments when the agent configuration or accessible fields change.

- Use Salesforce deployment validation before making significant metadata changes.

7. TESTING AND VALIDATION

7.1 Test Case 1 - Salesforce Data Retrieval

The Salesforce CLI data query for Parcel ID P-001 returned one record. The record was FUTECH Laptop Package with Status In Transit, Weight 3, and Estimated Delivery 2026-05-10. This verifies that the local Salesforce DX project can connect to the Developer Org and query the intended Parcel data.

Test ID	Input	Expected Result	Observed Result	Status
TC-01	P-001	Matching parcel returned	FUTECH Laptop Package / P-001 / In Transit / 3 / 2026-05-10	PASS
TC-02	Agent query for P-001	Agent routes to Parcel Tracking and returns parcel data	Final AgentForce preview produced grounded parcel information	PASS
TC-03	Metadata deployment validation	All project metadata validates successfully	29/29 components validated, no test failures	PASS

7.2 Test Case 2 - AgentForce Final Preview

The final submitted AgentForce preview shows the query for parcel P-001 and a grounded response. The trace displayed the flow from input through transition to the Parcel Tracking subagent, the Get Parcel Details action, and output evaluation.

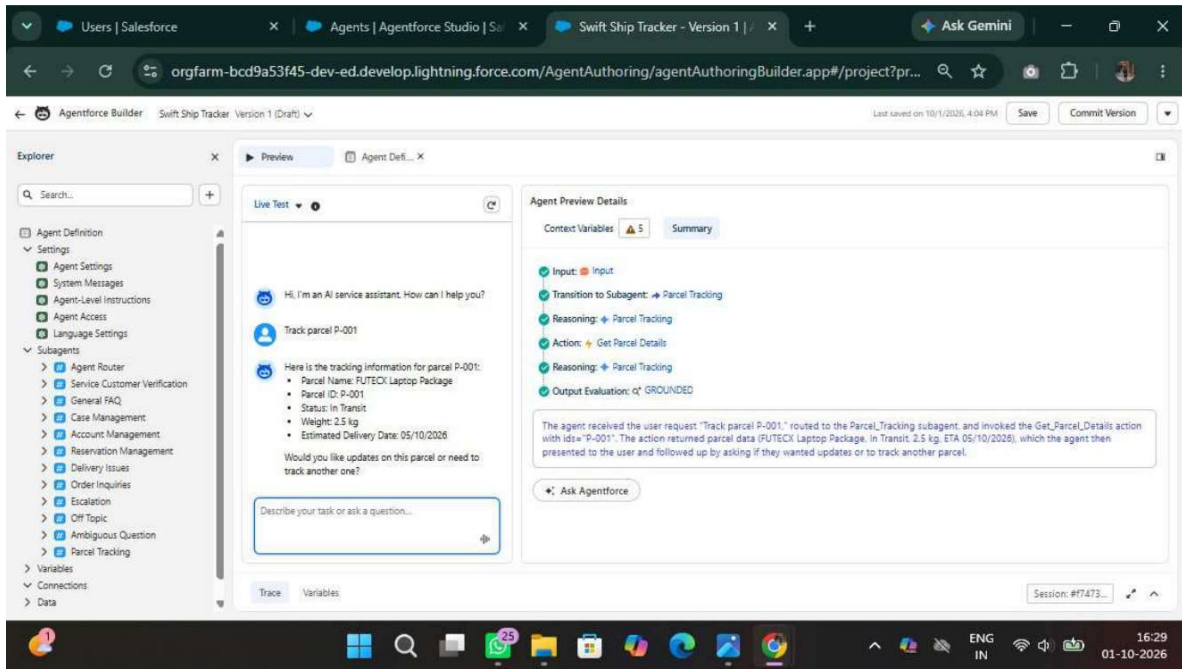


Figure 23. Final Swift Ship Tracker Agent preview and reasoning trace for the parcel-tracking interaction.

7.3 Deployment Validation

The Salesforce CLI deployment validation completed successfully for 29/29 metadata components. The validated set included the custom objects and fields, Parcel_Details Flow, Retrieve_Parcel_Details Prompt Template, list views, and Swift_Ship Permission Set. No automated tests were executed during this metadata validation because the validation run reported zero tests.

7.4 Source Management Validation

- Git repository initialized on the main branch.
- Commit author updated to Ashwin Ramakrishnan.
- Remote configured as the GitHub repository AshwinRamakrishnan/swift-ship-tracker.
- Local branch synchronized with origin/main.
- Working tree verified clean after the final source push.

GitHub Repository: <https://github.com/AshwinRamakrishnan/swift-ship-tracker>

8. CONCLUSION

Swift Ship Tracker demonstrates how Salesforce can be used as a centralized parcel-management platform combined with conversational AI for tracking. The implementation establishes a structured data layer using Parcel, Delivery, Sender, and Receiver objects and connects that data to Flow automation, Prompt Builder, and AgentForce.

The implemented Parcel Tracking workflow provides a concrete end-to-end path from a user query to a Salesforce record and back to a grounded conversational response. The current project evidence shows the required objects and fields, project-specific sample records, Prompt Builder execution, Flow configuration, permission setup, AgentForce subagent and action configuration, and final AgentForce preview.

The project also demonstrates source management and deployment discipline. The Salesforce metadata was validated through the Salesforce CLI, and the local project source was synchronized to GitHub under the AshwinRamakrishnan account. The result is a documented academic prototype that can be extended with additional automation, analytics, customer self-service, and external logistics integrations.

9. FUTURE SCOPE

- Extend AgentForce to answer a wider range of parcel and delivery questions.
- Add additional automated status-update and notification workflows.
- Expand customer self-service through Experience Cloud.
- Introduce more advanced delivery-performance reports and dashboards.
- Integrate external logistics or map services for live tracking.
- Add stronger test automation and CI/CD pipelines for larger deployments.
- Improve prompt quality and conversation handling for exception cases and missing data.

10.REFERENCES

[1] Swift Ship Tracker project reference document supplied for the university project implementation.

[2] Salesforce Developer Org used for the Swift Ship Tracker implementation and validation.

[3] Salesforce CLI project source maintained in the GitHub repository:

<https://github.com/AshwinRamakrishnan/swift-ship-tracker>

11.APPENDIX

- A. Custom Objects: Parcel, Delivery, Sender, Receiver
- B. Standard Objects: Account, Contact, Case, Task, EmailMessage, User
- C. Automation: Flows, Approval Processes, Email Alerts
- D. Apex: Apex Classes, Batch Apex
- E. AI: AgentForce AI, Prompt Builder
- F. UI: Lightning App, Lightning Record Pages, Experience Cloud
- G. Security: Profiles, Roles, Permission Sets, Sharing Rules, Field-Level Security
- H. Analytics: Reports & Dashboards
- I. Testing: Flow, AgentForce, data retrieval, security and permission validation

Thank You